Week 3 Module

Merge Sort

Merge Sort is a divide and conquer sorting algorithm that works by repeatedly splitting a list into smaller sublists, sorting those sublists, and then merging them back together in order.

Merge Sort uses recursion to divide the list into smaller parts.

It's an efficient sorting algorithm for large datasets. It's widely used due to its predictable performance.

The time complexity of Merge Sort is $O(n \log n)$.

It requires additional memory during merging process and works well even when the input data is already sorted.

Week 3 Module:

Date: 29/06/2026

Topic 1: Merge Sort

Question 1 (Easy) – Rank Students by Marks

A school has recently conducted a scholarship examination for all students of Class 12. The examination department has collected the marks of all students in an array. Due to the large number of participants, manually arranging the marks in ascending order has become difficult.

Your task is to write a program that takes the marks of N students and sorts them in ascending order using the Merge Sort algorithm.

Merge Sort works by repeatedly dividing the array into smaller subarrays until each subarray contains a single element. These smaller subarrays are then merged in a sorted manner to produce the final sorted array.

The school administration wants the marks to be displayed in increasing order so they can easily identify the lowest and highest scoring students and generate performance reports.

Input

- **First line contains an integer N, representing the number of students.**
- **Second line contains N space-separated integers, representing the marks of the students.**

Output

- **Print the marks in ascending order.**

Constraints

- **1 ≤ N ≤ 10^5**
- **0 ≤ Marks ≤ 100**

Example

Input:

6
78 45 92 34 67 81

Output:

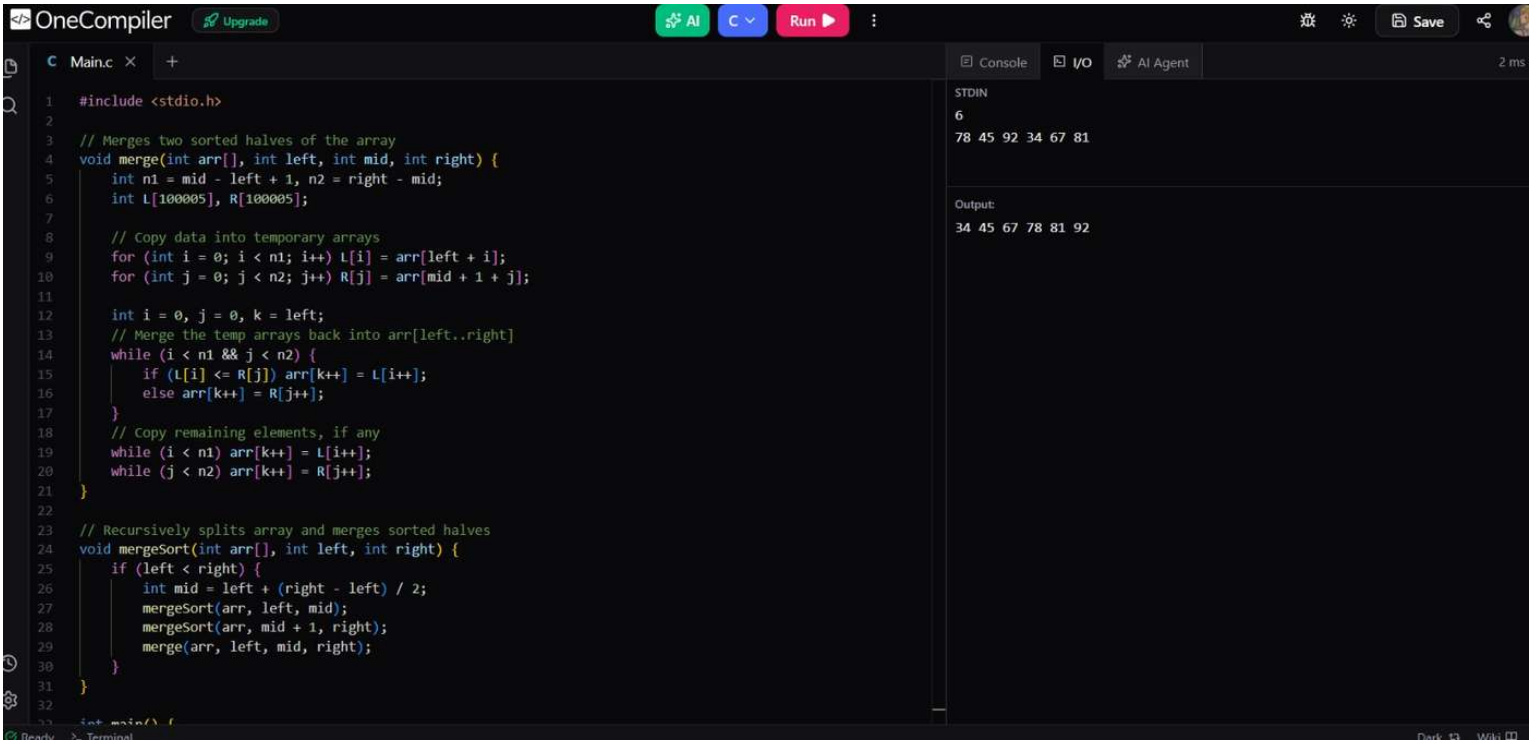
34 45 67 78 81 92

1⇒ Rank students by marks

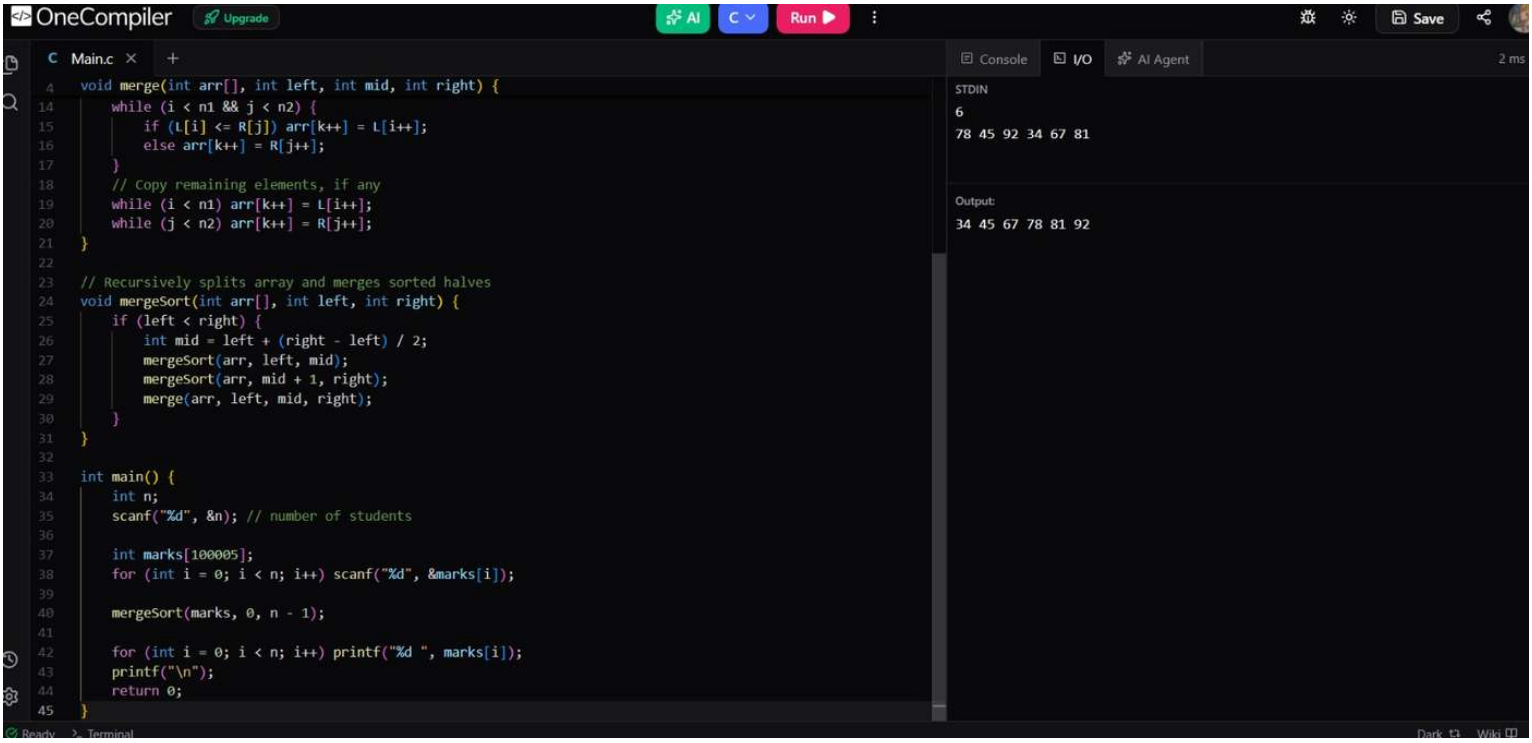
```
#include <stdio.h>
void merge(int arr[], int left, int mid, int right){
int n1 = mid - left + 1, n2 = right - mid;
int L[100005], R[100005];
for (int i = 0; i < n1; i++) L[i] = arr[left + i];
for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
int i = 0, j = 0, K = left;
while (i < n1 && j < n2){
if(L[i] <= R[j]) arr[K++] = L[i++];
else arr[K++] = R[j++];
}
while(i < n1) arr[K++] = L[i++];
while(j < n2) arr[K++] = R[j++];
}
 void mergeSort(int arr[], int left, int right){
  if(left < right){
   int mid = left + (right - left)/2;
  mergeSort(arr, left, mid);
  mergeSort(arr, mid + 1, right);
  merge(arr, left, mid, right);
  }
 }
 int main(){
  int n;
 Scanf("%d", &n);
 int marks[100005];
 for(int i = 0; i < n; i++) scanf("%d", &marks[i]);
 mergeSort(marks, 0, n-1);
 for(int i = 0; i < n; i++) Printf("%d", marks[i]);
 Print f("\n");
 return 0;
}
```

Input: 6
78 45 92 34 67 81

Output: 34 45 67 78 81 92




```c
#include <stdio.h>

// Merges two sorted halves of the array
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1, n2 = right - mid;
    int L[100005], R[100005];

    // Copy data into temporary arrays
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    // Merge the temp arrays back into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    // Copy remaining elements, if any
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

// Recursively splits array and merges sorted halves
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
```




```c
void merge(int arr[], int left, int mid, int right) {
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    // Copy remaining elements, if any
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

// Recursively splits array and merges sorted halves
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    scanf("%d", &n); // number of students

    int marks[100005];
    for (int i = 0; i < n; i++) scanf("%d", &marks[i]);

    mergeSort(marks, 0, n - 1);

    for (int i = 0; i < n; i++) printf("%d ", marks[i]);
    printf("\n");
    return 0;
}
```

Question 2 (Easy) – Organizing Product Prices

An e-commerce company stores product prices in the order they are added to the platform. Before publishing a seasonal sales report, the company wants all product prices to be arranged in ascending order.

Since the number of products can be very large, an efficient sorting technique is required. Your task is to implement Merge Sort to arrange the prices from the cheapest product to the most expensive product.

The program should divide the list into smaller parts, sort each part, and merge them back together while maintaining the correct order.

Input

- **First line contains integer N.**
- **Second line contains N space-separated product prices.**

Output

- **Print the sorted prices in ascending order.**

Constraints

- **$1 \leq N \leq 100000$**
- **$1 \leq Price \leq 1000000$**

Example

Input:

5
2500 1500 4000 3000 1000

Output:

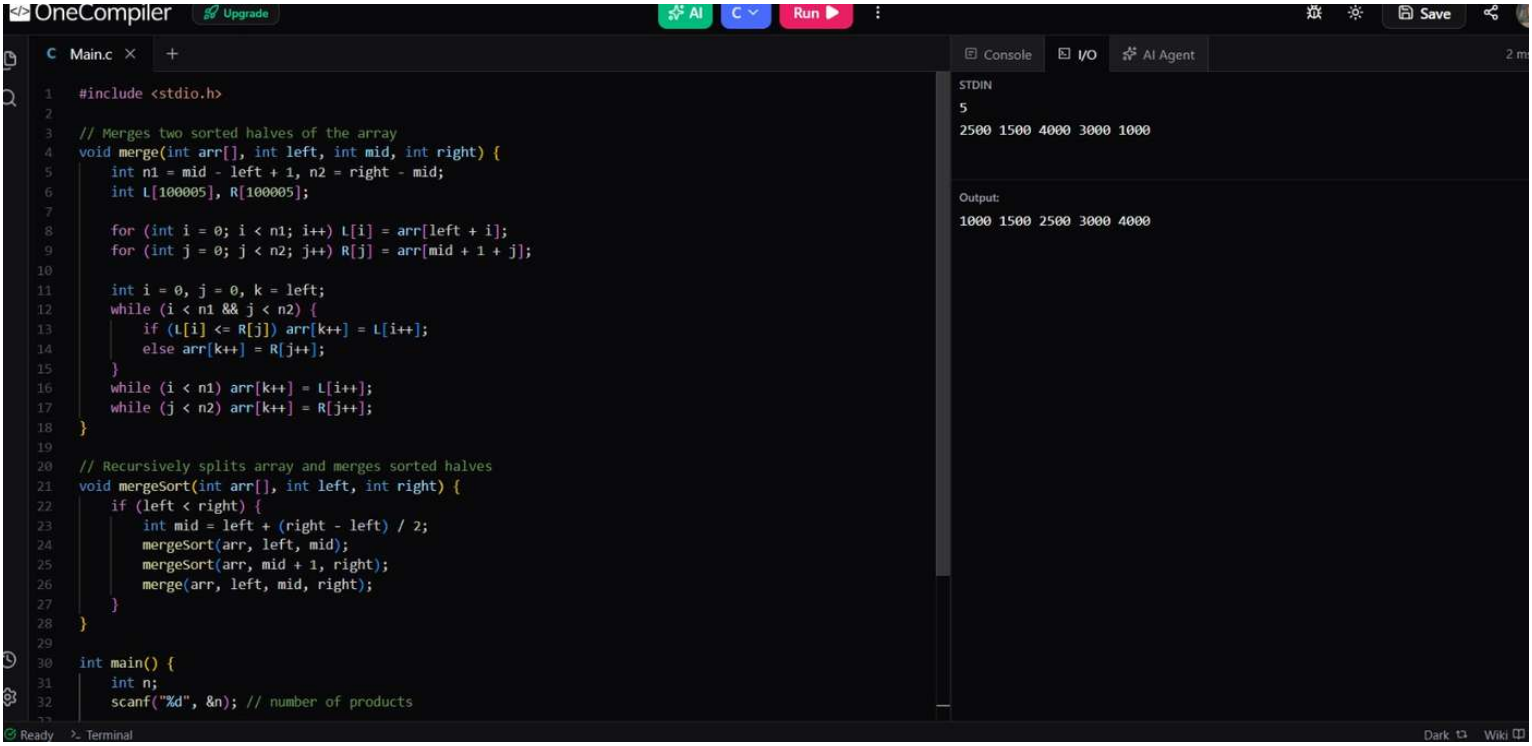
1000 1500 2500 3000 4000

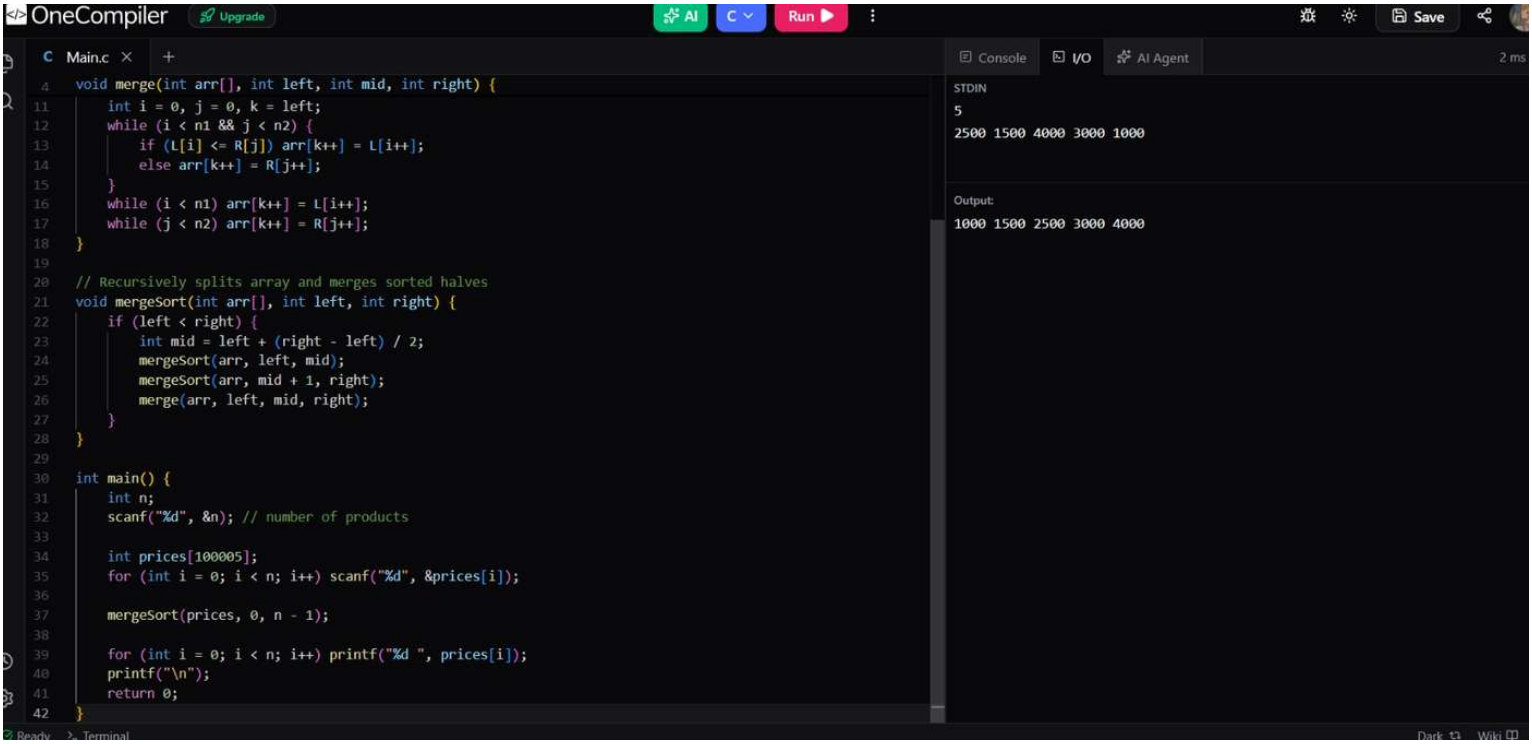
2→ Organizing Product Prices

```
#include <stdio.h>
void merge(int arr[], int left, int mid, int right)
{
 int n1 = mid - left + 1, n2 = right - mid;
 int L[100005] R[100005];
for (int i = 0; i < n1; i++) L[i] = arr[left+i];
for(int j = 0; j < n2; j++) R[j] = arr[mid+1+j];
 int i = 0, j = 0, K = left;
while (i < n1 && j < n2) {
  if (L[i] <= R[j]) arr[K++] = L[i++];
 else arr[K++] = R[j++];
}
 while (i < n1) arr[K++] = L[i++];
 while(j < n2) arr[K++] = R[j++];
}
 void mergeSort (int arr[], int left, int right) {
   if (left < right) {
    int mid = left + (right - left) / 2;
  mergeSort (arr, left, mid);
  mergeSort (arr, mid + 1, right);
  merge (arr, left, mid, right);
 }
}
 int main() {
 int n;
 Scanf ("%d", &n);
 int prices[100005];
for(int i=0; i<n; i++) scanf ("%d", &prices[i]);
 mergeSort (prices, 0, n-1);
for(int i = 0; i<n; i++) print f("%d", prices[i]);
 print f ("\n");
 return 0;
}
```

Input : 5
2500 1500 4000 3000 1000

Output : 1000 1500 2500 3000 4000


OneCompiler
Upgrade
AI
C
Run
Save
Main.c
Console
I/O
AI Agent
STDIN
5
2500 1500 4000 3000 1000
Output:
1000 1500 2500 3000 4000
#include <stdio.h>
// Merges two sorted halves of the array
void merge(int arr[], int left, int mid, int right) {
int n1 = mid - left + 1, n2 = right - mid;
int L[100005], R[100005];
for (int i = 0; i < n1; i++) L[i] = arr[left + i];
for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
if (L[i] <= R[j]) arr[k++] = L[i++];
else arr[k++] = R[j++];
}
while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];
}
// Recursively splits array and merges sorted halves
void mergeSort(int arr[], int left, int right) {
if (left < right) {
int mid = left + (right - left) / 2;
mergeSort(arr, left, mid);
mergeSort(arr, mid + 1, right);
merge(arr, left, mid, right);
}
}
int main() {
int n;
scanf("%d", &n); // number of products
Ready
Terminal
Dark
Wiki




```c
void merge(int arr[], int left, int mid, int right) {
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

// Recursively splits array and merges sorted halves
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    scanf("%d", &n); // number of products

    int prices[100005];
    for (int i = 0; i < n; i++) scanf("%d", &prices[i]);

    mergeSort(prices, 0, n - 1);

    for (int i = 0; i < n; i++) printf("%d ", prices[i]);
    printf("\n");
    return 0;
}
```

Question 3 (Medium) – Sort Employee Records by Salary

A multinational company has employee salary records stored in an array. Before generating annual reports, the Human Resources department wants the salary records arranged in ascending order.

The company expects the number of employees to grow significantly over time, so an efficient sorting algorithm must be used. You are required to implement Merge Sort and display the salaries in increasing order.

Additionally, after sorting, the company wants to know:

1. **The lowest salary.**
2. **The highest salary.**
3. **The difference between the highest and lowest salary.**

Input

- **First line contains integer N.**
- **Second line contains N space-separated salaries.**

Output

- **First line: Sorted salaries.**
- **Second line: Lowest salary.**
- **Third line: Highest salary.**
- **Fourth line: Salary difference.**

Constraints

- **$1 \leq N \leq 100000$**
- **$1000 \leq Salary \leq 1000000$**

Example

Input:

7
45000 25000 70000 50000 30000 90000 60000

Output:

25000 30000 45000 50000 60000 70000 90000
Lowest Salary: 25000
Highest Salary: 90000
Difference: 65000

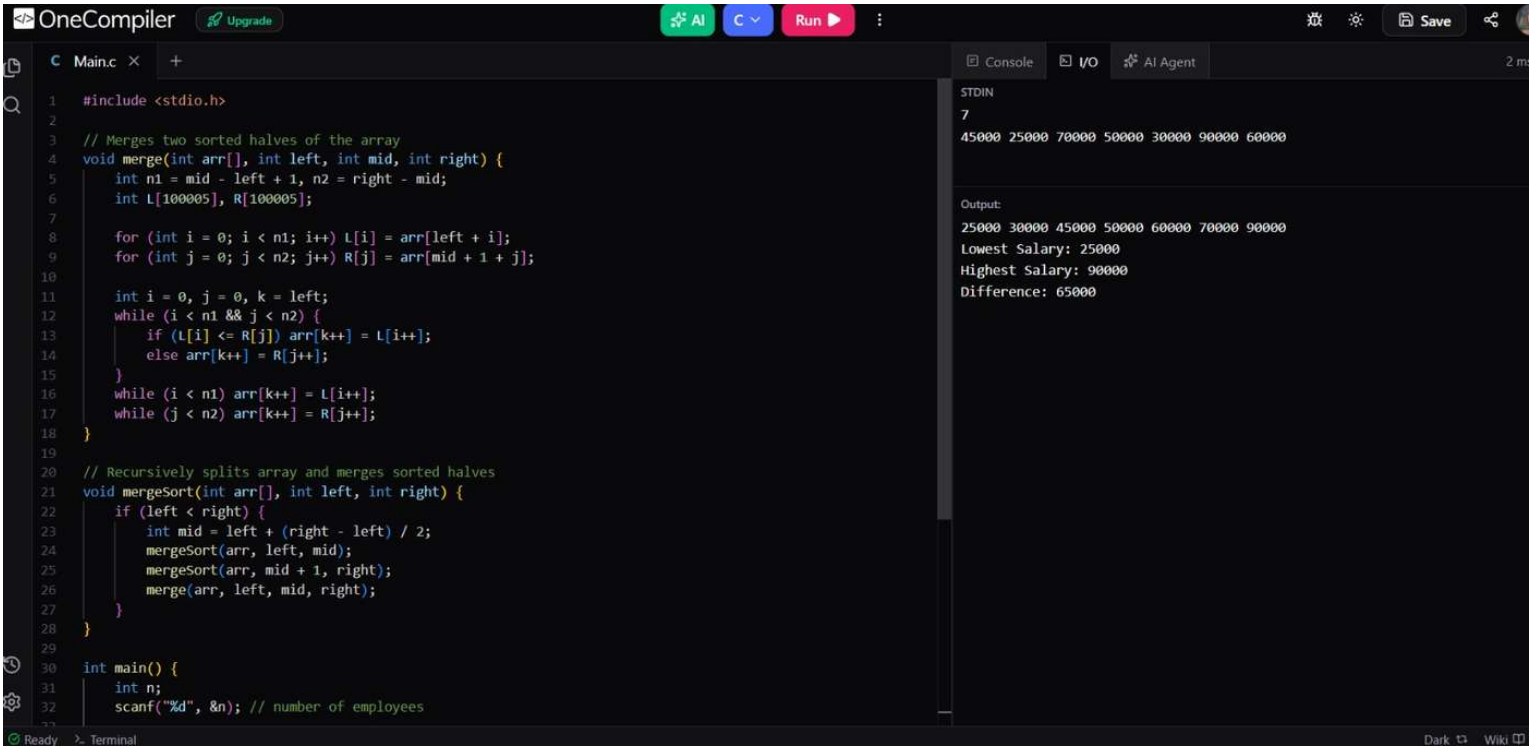
3⇒ Sort Employee Records by Salary

```
#include <stdio.h>
void merge(int arr[], int left, int mid, int right){
  int n1 = mid - left + 1, n2 = right - mid;
  int L[100005], R[100005];
  for(int i = 0; i < n1; i++) L[i] = arr[left + i];
  for(int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
  int i = 0, j = 0, K = left;
  while (i < n1 && j < n2) {
    if(L[i] <= R[j]) arr[K++] = L[i++];
    else arr[K++] = R[j++];
  }
  while(i < n1) arr[K++] = L[i++];
  while(j < n2) arr[K++] = R[j++];
}
void mergeSort(int arr[], int left, int right){
  if(left < right){
    int mid = left + (right - left)/2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid+1, right);
    merge(arr, left, mid, right);
  }
}
int main(){
  int n;
  Scanf("%d", &n);
  int salaries[10000 5];
  for(int i=0; i<n; i++) Scanf("%d", &salaries[i]);
  mergeSort(salaries, 0, n-1);
  for(int i=0; i<n; i++) printf("%d", salaries[i]);
  Printf("\n");
  Printf("Lowest Salary: %d\n", salaries[0]);
```

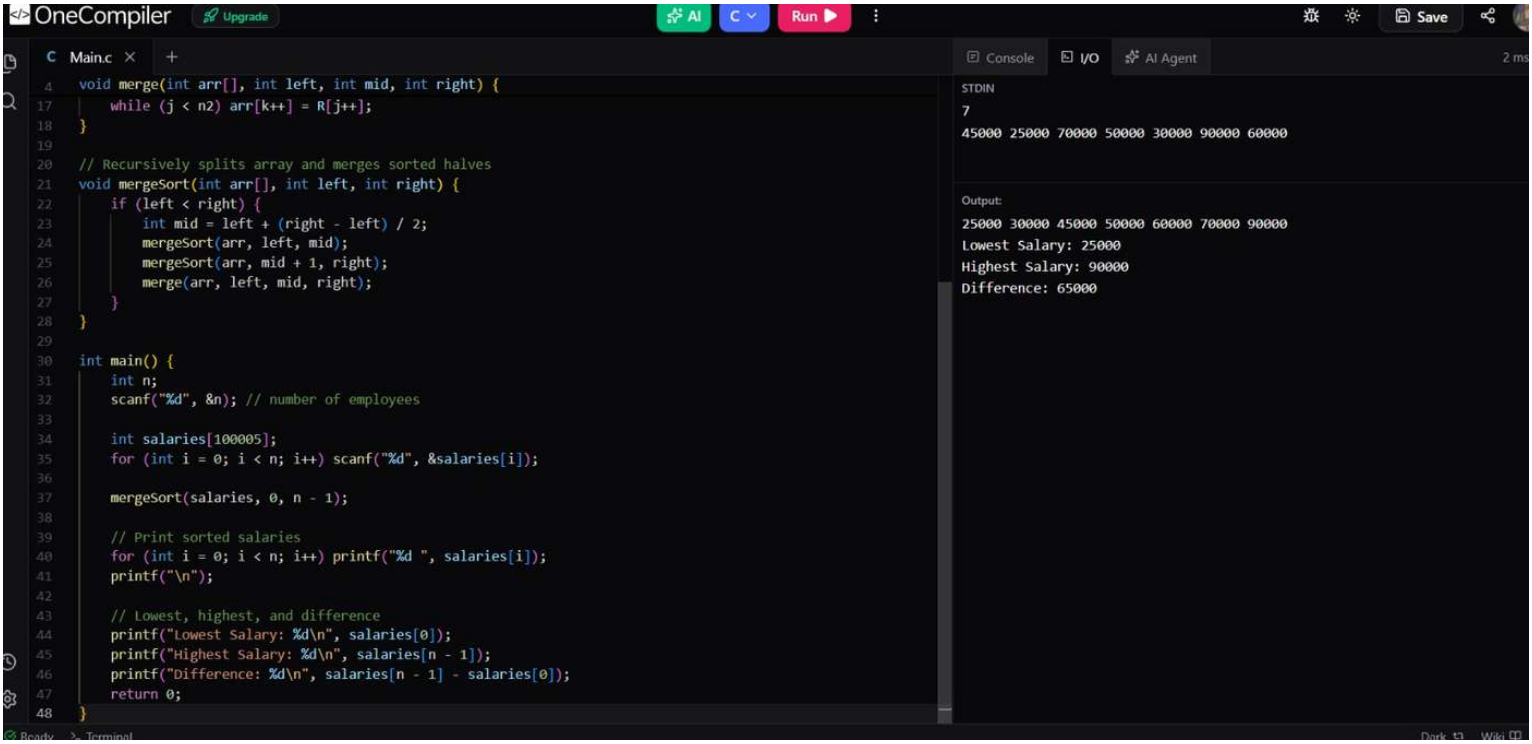
```
Print f ("Highest Salary : %d \n", salaries[n-1]);
Print f ("Difference: %d \n", salaries[n-1]-salaries[0]);
return 0;
}
```

Input : 7
45000 25000 70000 50000 30000 90000 60000

Output : 25000 30000 45000 50000 60000 70000 90000
Lowest Salary ⇒ 25000
Highest Salary ⇒ 90000
Difference ⇒ 65000




```c
#include <stdio.h>

// Merges two sorted halves of the array
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1, n2 = right - mid;
    int L[100005], R[100005];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

// Recursively splits array and merges sorted halves
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    scanf("%d", &n); // number of employees
```




```c
void merge(int arr[], int left, int mid, int right) {
    while (j < n2) arr[k++] = R[j++];
}

// Recursively splits array and merges sorted halves
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    scanf("%d", &n); // number of employees

    int salaries[100005];
    for (int i = 0; i < n; i++) scanf("%d", &salaries[i]);

    mergeSort(salaries, 0, n - 1);

    // Print sorted salaries
    for (int i = 0; i < n; i++) printf("%d ", salaries[i]);
    printf("\n");

    // Lowest, highest, and difference
    printf("Lowest Salary: %d\n", salaries[0]);
    printf("Highest Salary: %d\n", salaries[n - 1]);
    printf("Difference: %d\n", salaries[n - 1] - salaries[0]);
    return 0;
}
```

※ Quick Sort

Quick Sort is a divide and conquer sorting algorithm that works by repeatedly partitioning an array into smaller sub-arrays and sorting them.

It's fastest and most widely used sorting techniques.

The average time complexity of Quick Sort is $O(n \log n)$. The worst-case time complexity of Quick Sort is $O(n^2)$ when the pivot selection is unbalanced.

It requires less extra memory than Merge Sort.

It's not a stable sorting algorithm.

Topic 2: Quick Sort

Question 1 (Easy) – Race Timing Analysis

A sports academy records the completion times (in seconds) of runners participating in a race. The coach wants to arrange the timings from the fastest runner to the slowest runner.

To process large datasets efficiently, the academy has decided to use the Quick Sort algorithm. Your task is to sort the race timings in ascending order.

Input

- **First line contains integer N.**
- **Second line contains N space-separated timings.**

Output

- **Print the timings in ascending order.**

Constraints

- **$1 \leq N \leq 100000$**
- **$1 \leq Time \leq 10000$**

Example

Input:

6
55 49 62 58 45 51

Output:

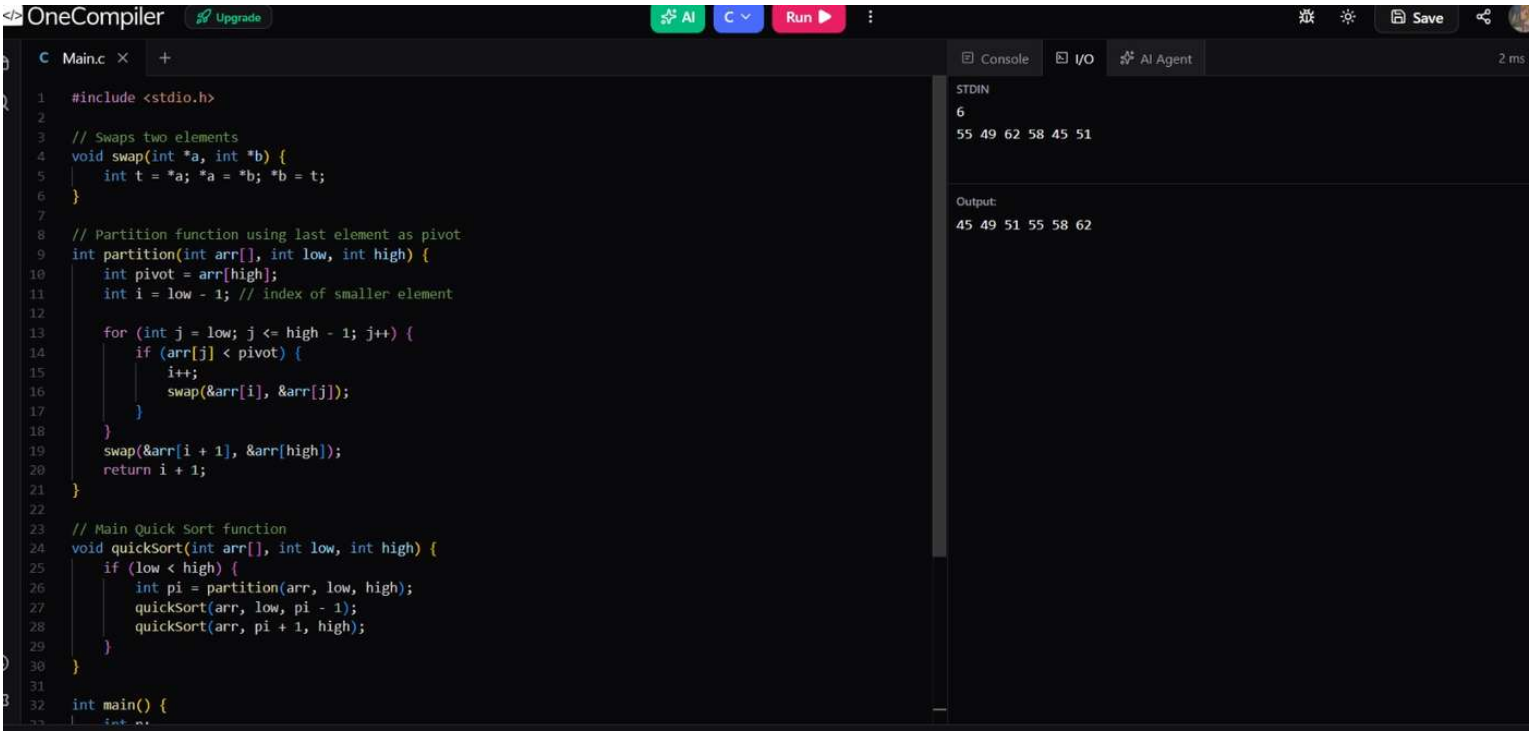
45 49 51 55 58 62

1⇒ Race Timing Analysis

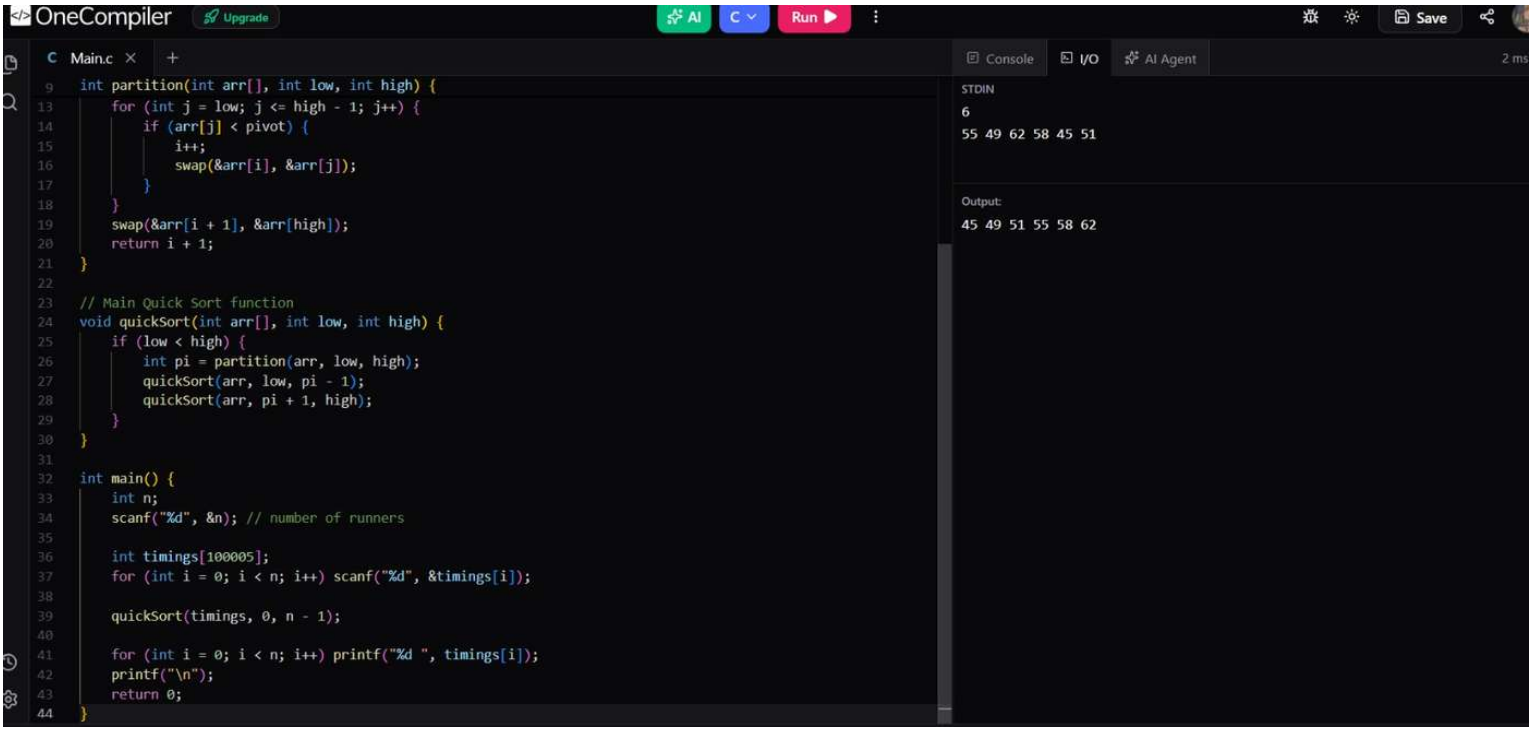
```
#include<stdio.h>
void swap(int *a, int *b){
  int t = *a; *a = *b; *b = t;
}
int partition(int arr[], int low, int high){
int pivot = arr[high];
int i = low - 1;
for(int j = low; j <= high-1; j++){
if(arr[j] < pivot){
 i++;
 swap(&arr[i], &arr[j];
}
}
}
swap(&arr[i+1], &arr[high]);
return i+1;
}
void quickSort(int arr[], int low, int high){
 if(low < high){
 int pi = partition(arr, low, high);
quickSort(arr, low, pi-1);
quickSort(arr, pi+1, high);
}
}
 int main(){
  int n;
Scanf("%d", &n);
 int timings[100005];
 for(int i=0; i<n; i++)Scanf("%d", &timings[i]);
quickSort(timings, 0, n-1);
for(int i=0; i<n; i++) printf("%d", timings[i]);
Print f("\n");
return 0;
}
```

Input : 6
55 49 62 58 45 51

Output : 45 49 51 55 58 62




```c
#include <stdio.h>

// Swaps two elements
void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

// Partition function using last element as pivot
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1; // index of smaller element

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Main Quick Sort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
```


OneCompiler
Upgrade
AI
C
Run
Save
Main.c
Console
I/O
AI Agent
2 ms
int partition(int arr[], int low, int high) {
for (int j = low; j <= high - 1; j++) {
if (arr[j] < pivot) {
i++;
swap(&arr[i], &arr[j]);
}
}
swap(&arr[i + 1], &arr[high]);
return i + 1;
}
// Main Quick Sort function
void quickSort(int arr[], int low, int high) {
if (low < high) {
int pi = partition(arr, low, high);
quickSort(arr, low, pi - 1);
quickSort(arr, pi + 1, high);
}
}
int main() {
int n;
scanf("%d", &n); // number of runners
int timings[100005];
for (int i = 0; i < n; i++) scanf("%d", &timings[i]);
quickSort(timings, 0, n - 1);
for (int i = 0; i < n; i++) printf("%d ", timings[i]);
printf("\n");
return 0;
}
STDIN
6
55 49 62 58 45 51
Output:
45 49 51 55 58 62

Question 2 (Easy) – Organizing Library Book IDs

A library stores books using unique identification numbers. Due to years of additions and removals, the IDs are no longer arranged properly.

The librarian wants all IDs sorted in ascending order before migrating the data to a new management system. Implement Quick Sort to arrange the IDs efficiently.

Input

- **First line contains integer N.**
- **Second line contains N space-separated book IDs.**

Output

- **Print the sorted IDs.**

Constraints

- **1 ≤ N ≤ 100000**
- **1 ≤ ID ≤ 10^9**

Example

Input:

5
1025 1001 1050 1010 1005

Output:

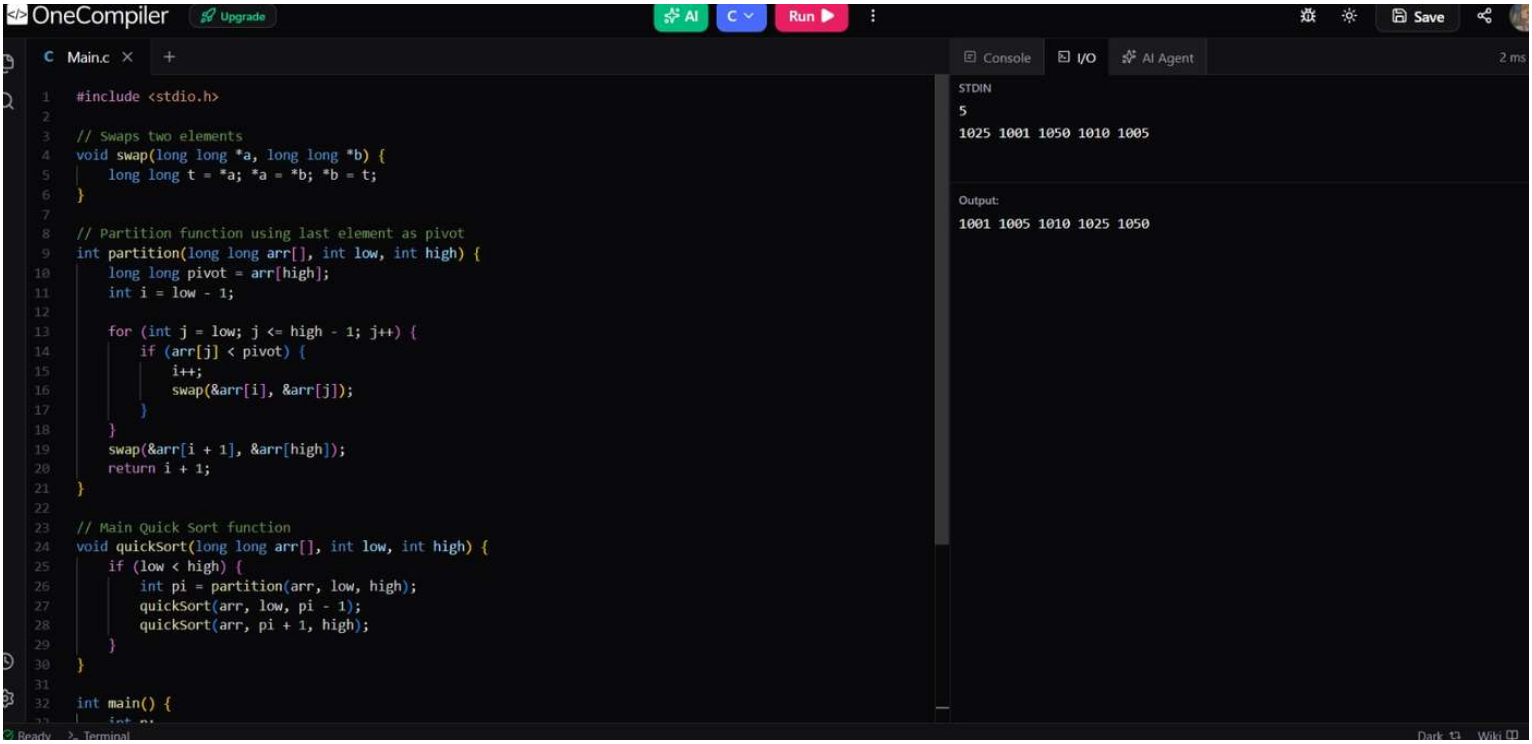
1001 1005 1010 1025 1050

2⇒ Organizing Library Book IDs

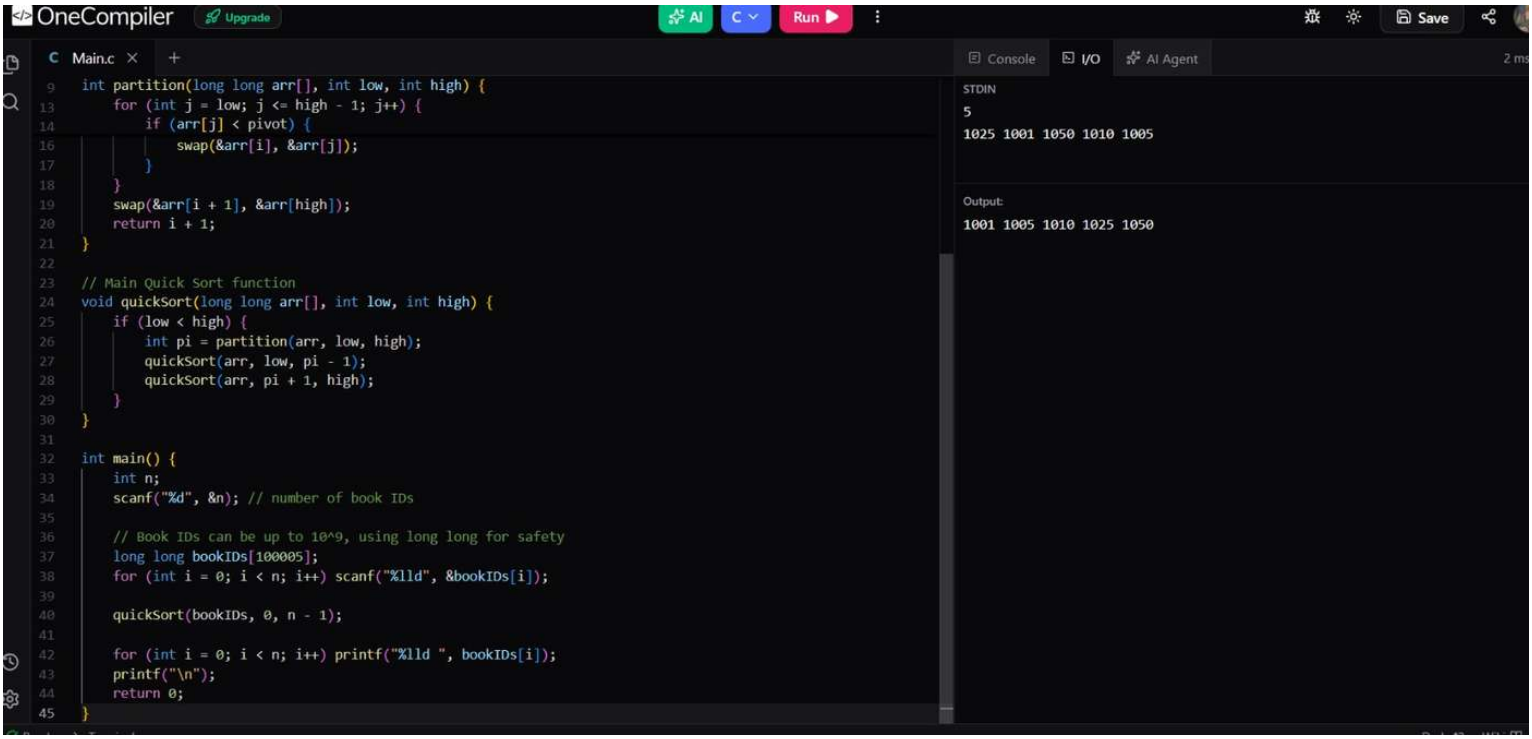
```
#include <stdio.h>
void swap(long long *a, long long *b) {
 long long t = *a; *a = *b; *b = t;
}
int partition(long long arr[], int low, int high) {
long long pivot = arr[high];
int i = low - 1;
for (int j = low; j <= high - 1; j++) {
 if (arr[j] < pivot) {
  i++;
  swap(&arr[i], &arr[j]);
 }
}
swap(&arr[i+1], &arr[high]);
return i+1;
}
void quickSort(long long arr[], int low, int high) {
if (low < high) {
 int pi = partition(arr, low, high);
 quickSort(arr, low, pi-1);
 quickSort(arr, pi+1, high);
 }
}
 int main() {
  int n;
Scanf("%d", &n);
long long bookIDs[100005];
for(int i=0; i<n; i++) scanf("%lld", &bookIDs[i]);
quickSort(bookIDs, 0, n-1);
for(int i=0; i<n; i++) Printf("%lld", bookIDs[i]);
Printf("\n");
return 0;
}
```

Input: 5
1025 1001 1050 1010 1005

Output: 1001 1005 1010 1025 1050




```c
#include <stdio.h>

// Swaps two elements
void swap(long long *a, long long *b) {
    long long t = *a; *a = *b; *b = t;
}

// Partition function using last element as pivot
int partition(long long arr[], int low, int high) {
    long long pivot = arr[high];
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Main Quick Sort function
void quickSort(long long arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
```


OneCompiler
Upgrade
AI
C
Run
Save
Main.c
Console
I/O
AI Agent
STDIN
5
1025 1001 1050 1010 1005
Output:
1001 1005 1010 1025 1050
int partition(long long arr[], int low, int high) {
for (int j = low; j <= high - 1; j++) {
if (arr[j] < pivot) {
swap(&arr[i], &arr[j]);
}
}
swap(&arr[i + 1], &arr[high]);
return i + 1;
}
// Main Quick Sort function
void quickSort(long long arr[], int low, int high) {
if (low < high) {
int pi = partition(arr, low, high);
quickSort(arr, low, pi - 1);
quickSort(arr, pi + 1, high);
}
}
int main() {
int n;
scanf("%d", &n); // number of book IDs
// Book IDs can be up to 10^9, using long long for safety
long long bookIDs[100005];
for (int i = 0; i < n; i++) scanf("%lld", &bookIDs[i]);
quickSort(bookIDs, 0, n - 1);
for (int i = 0; i < n; i++) printf("%lld ", bookIDs[i]);
printf("\n");
return 0;
}

Question 3 (Medium) – Online Store Sales Ranking

An online marketplace tracks the daily sales made by different sellers. At the end of each week, management wants to identify the best-performing sellers.

You are given the total sales made by N sellers. Using Quick Sort, arrange the sales figures in descending order.

After sorting, display:

1. **Sorted sales figures.**
2. **Top 3 highest sales values.**
3. **Average of the top 3 sales values.**

Input

- **First line contains integer N.**
- **Second line contains N space-separated sales values.**

Output

- **Sorted sales figures in descending order.**
- **Top 3 sales values.**
- **Average of top 3 sales.**

Constraints

- **$3 \leq N \leq 100000$**
- **$1 \leq Sales \leq 1000000$**

Example

Input:

6
1200 5000 3000 2500 7000 4500

Output:

7000 5000 4500 3000 2500 1200
Top 3: 7000 5000 4500
Average: 5500

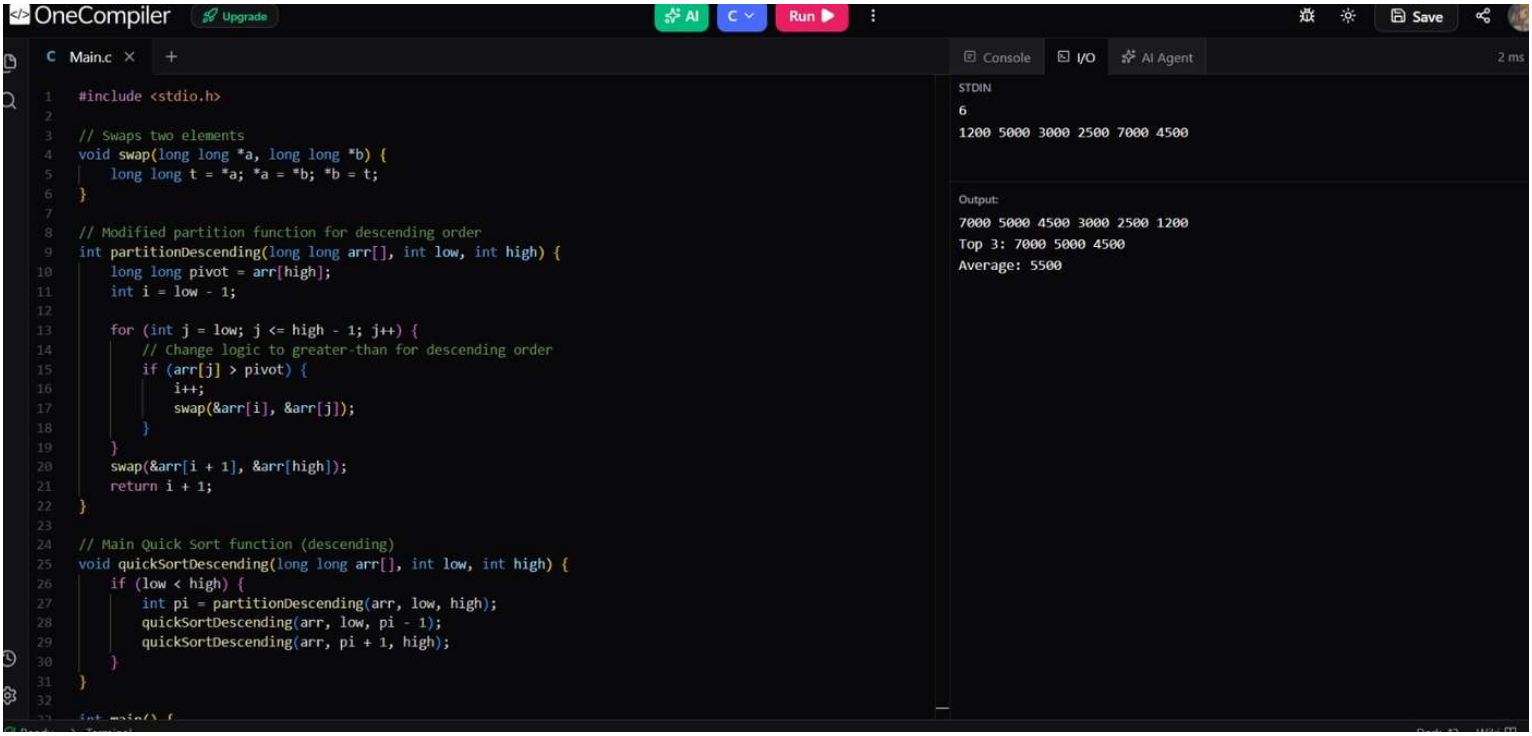
3⇒ Online Store Sales Ranking

```
#include <stdio.h>
void swap(long long *a, long long *b){
long long t = *a; *a = *b; *b=t;
}
int partitionDescending(long long arr[], int low, int high){
long long pivot = arr[high];
int i = low-1;
for(int j=low; j<=high-1; j++){
if(arr[j]>pivot){
i++;
swap(&arr[i], &arr[j]);
}
}
Swap(&arr[i+1], &arr[high]);
return i+1;
}
Void quickSortDescending(long long arr[], int low, int high){
if(low<high){
int pi = partitionDescending(arr, low, high);
quickSortDescending(arr, low, pi-1);
quickSortDescending(arr, pi+1, high);
}
}
int main(){
int n;
Scanf("%d", &n);
long long sales[100005];
for(int i=0; i<n; i++) scanf("%lld", &sales[i]);
quickSortDescending(sales, 0, n-1);
for(int i=0; i<n; i++) Printf("%lld", sales[i]);
Print f("\n");
```

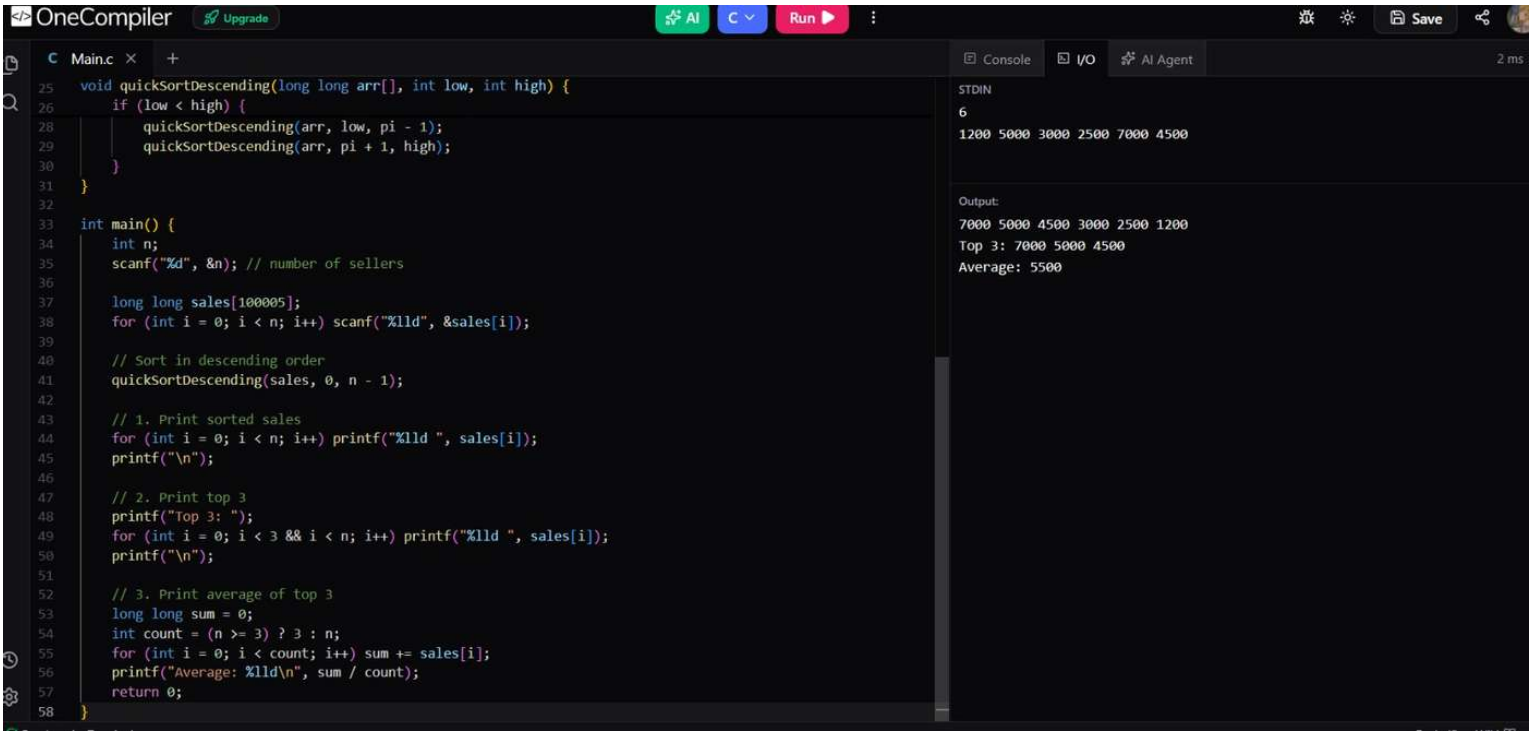
```
long long sum = 0;
int count = (n >= 3) ? 3 : n;
for(int i = 0; i < count; i++) sum += sales[i];
Printf("Average : %lld\n", sum / count);
return 0;
}
```

Input : 6
1200 5000 3000 2500 7000 4500

Output : 7000 5000 4500 3000 2500 1200
Top 3 ⇒ 7000 5000 4500
Average ⇒ 5500




```c
#include <stdio.h>

// Swaps two elements
void swap(long long *a, long long *b) {
    long long t = *a; *a = *b; *b = t;
}

// Modified partition function for descending order
int partitionDescending(long long arr[], int low, int high) {
    long long pivot = arr[high];
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        // Change logic to greater-than for descending order
        if (arr[j] > pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Main Quick Sort function (descending)
void quickSortDescending(long long arr[], int low, int high) {
    if (low < high) {
        int pi = partitionDescending(arr, low, high);
        quickSortDescending(arr, low, pi - 1);
        quickSortDescending(arr, pi + 1, high);
    }
}
```


OneCompiler
Upgrade
AI
C
Run
Save
Main.c
Console
I/O
AI Agent
2 ms
void quickSortDescending(long long arr[], int low, int high) {
if (low < high) {
quickSortDescending(arr, low, pi - 1);
quickSortDescending(arr, pi + 1, high);
}
}
int main() {
int n;
scanf("%d", &n); // number of sellers
long long sales[100005];
for (int i = 0; i < n; i++) scanf("%lld", &sales[i]);
// Sort in descending order
quickSortDescending(sales, 0, n - 1);
// 1. Print sorted sales
for (int i = 0; i < n; i++) printf("%lld ", sales[i]);
printf("\n");
// 2. Print top 3
printf("Top 3: ");
for (int i = 0; i < 3 && i < n; i++) printf("%lld ", sales[i]);
printf("\n");
// 3. Print average of top 3
long long sum = 0;
int count = (n >= 3) ? 3 : n;
for (int i = 0; i < count; i++) sum += sales[i];
printf("Average: %lld\n", sum / count);
return 0;
}
STDIN
6
1200 5000 3000 2500 7000 4500
Output:
7000 5000 4500 3000 2500 1200
Top 3: 7000 5000 4500
Average: 5500